Deep Learning Model Presentation

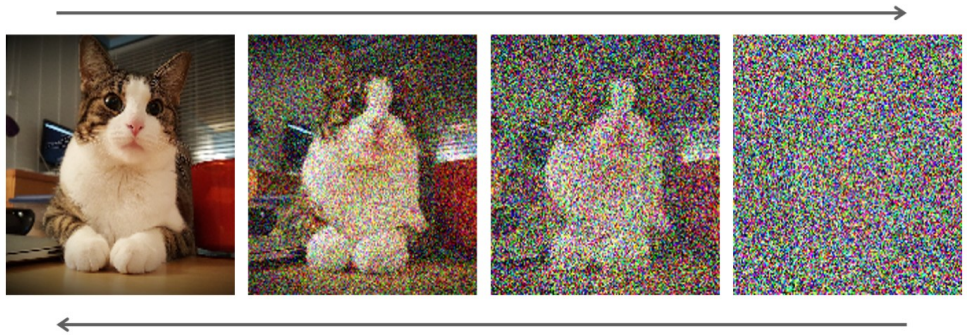
Diffusion Models

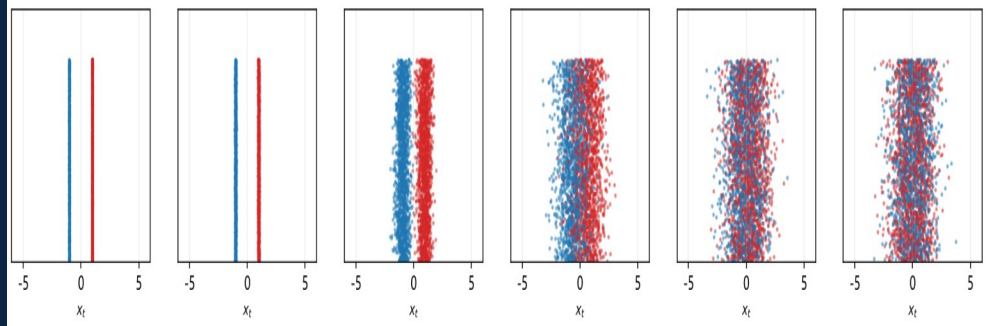
Learning to generate images by reversing a gradual noising process

Presented by:
WU Peihang
24108869D

$$x_t = \sqrt{(\bar{\alpha}_t)} \cdot x_0 + \sqrt{(1-\bar{\alpha}_t)} \cdot \varepsilon$$

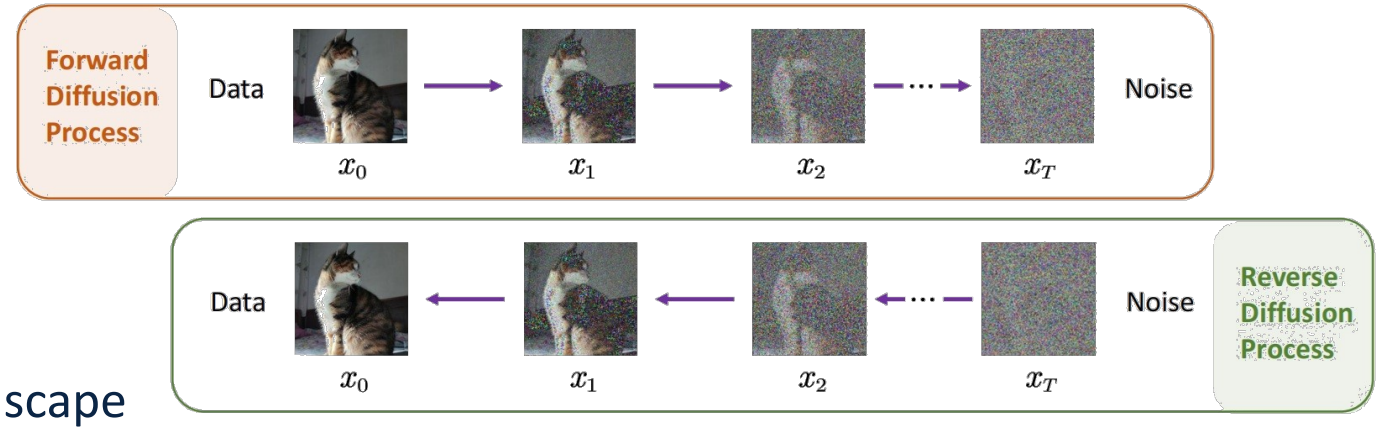




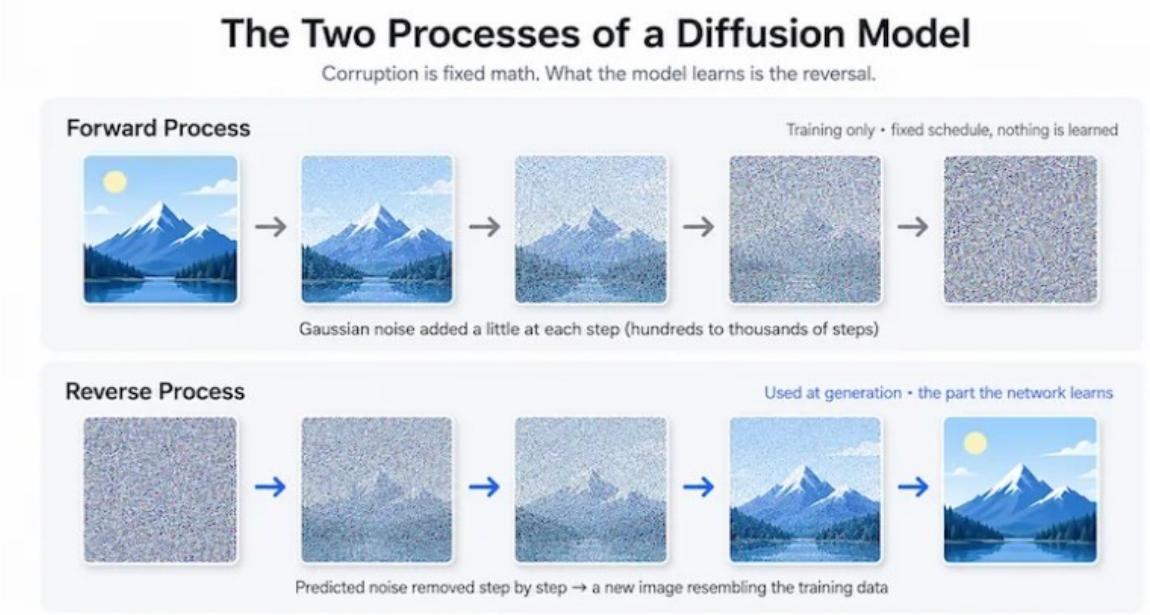


ROADMAP

Outline



1. Why diffusion? Generative modelling landscape
2. Forward process — the closed-form shortcut
3. Reverse process — the intractable posterior
4. Variational bound — deriving the training loss
5. Training and sampling algorithms
6. Design choices, results, and modern extensions



* The Denoising Diffusion Probabilistic Model is the variant discussed here.

Background: what is a diffusion model?

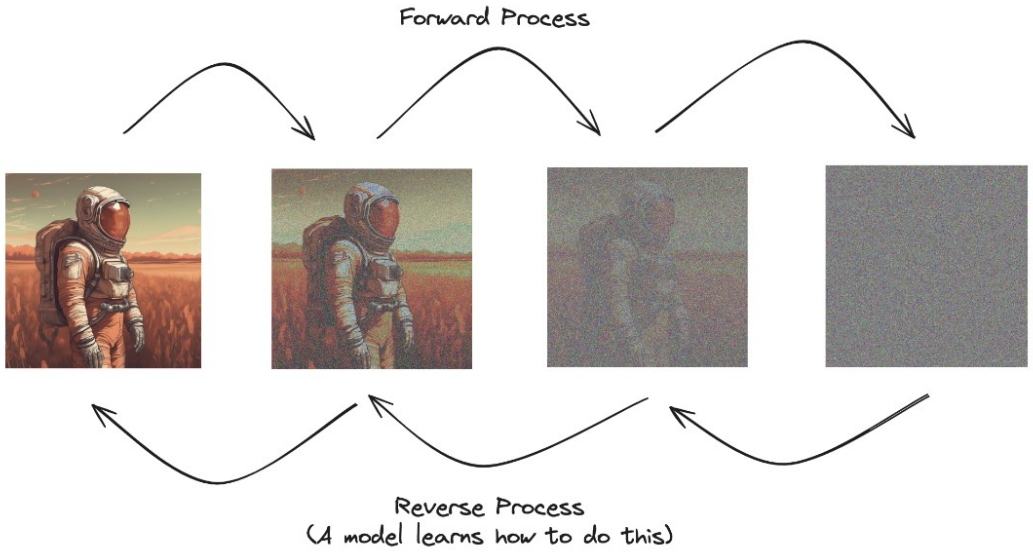
Diffusion models are inspired by a familiar physical process: particles drift from high to low concentration until the system reaches equilibrium.

A diffusion model borrows this idea. It adds noise to an image step by step, then trains a network to reverse that process.

Once the network can turn noise back into an image, we can generate new samples from pure random noise — no existing image required.

"Can Diffusion get reversed?"



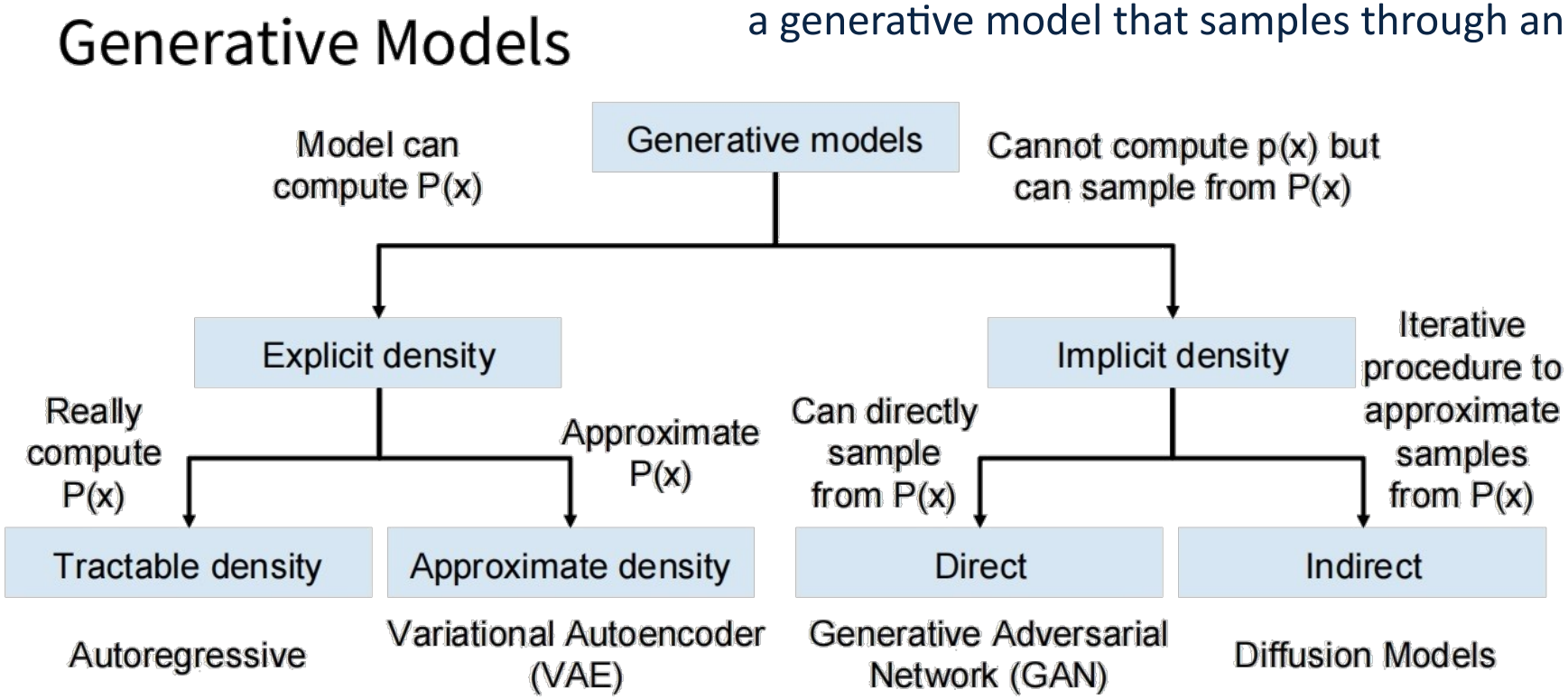


Where is the diffusion model?

An overview of the generative modelling landscape

Diffusion sits under Implicit density → Indirect sampling:

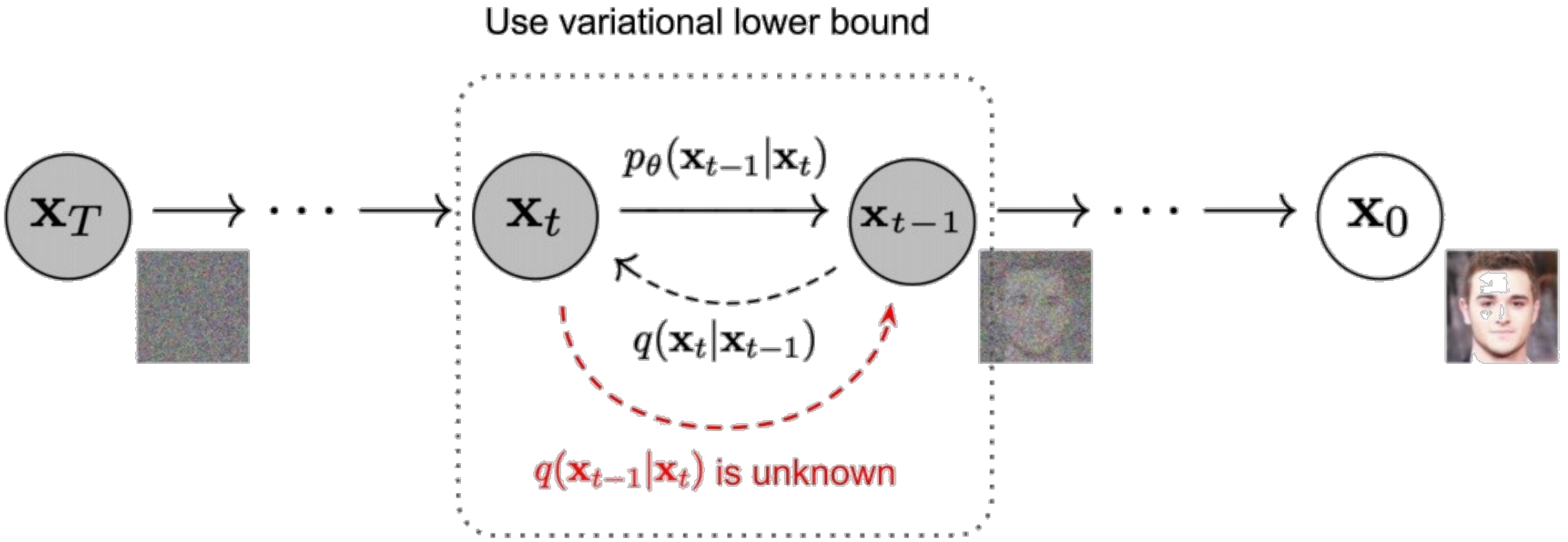
a generative model that samples through an indirect, iterative procedure.



Classification:
Implicit density
Indirect sampling

Figure adapted from Goodfellow, Tutorial on Generative Adversarial Networks, 2017

The forward and backward processes of Diffusion Model

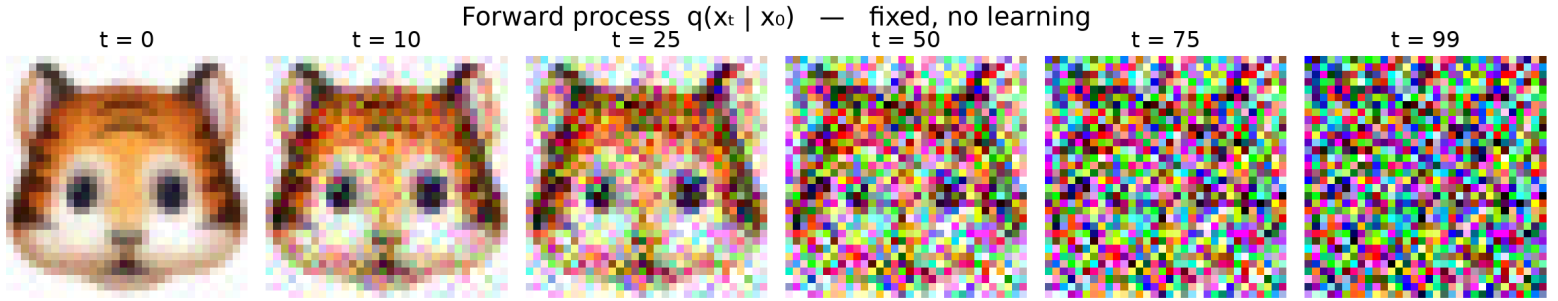


Diffusion models are bi-directional, comprising the noise addition process and the denoising process

The noise addition process follows the Markov Chain, easier to find the relationship between time steps.

The denoising process is similar to "entropy reduction", being more challenging to tackle.

The core idea: destroy information, then learn to restore it



Measured on a real training image (tiger face, 32×32): $q(x_t|x_0)$ over t = 0 → 99. Noise is Gaussian at every step and the schedule is fixed — nothing is learned here.

$$x_0 \to x_1 \to \cdots \to x_T \approx \mathcal{N}(0, I)$$

FORWARD fixed, no learning — gradually add Gaussian noise over T steps until the signal becomes pure noise

$$x_T \to \cdots \to x_1 \to x_0$$

REVERSE learned — train a network to undo one small noising step at a time

SAMPLE inference — iteratively denoise from random noise to obtain a new image

Key point: each step is small, so both the forward and the learned reverse step can be modelled as Gaussians.

SETUP

Notation

$\mathbf{x_0}$	clean data sample drawn from the data distribution $q(x_o)$
$\mathbf{x_t}$	noisy sample at time step t, t = 1 … T
$\boldsymbol{\beta_t \in (0, 1)}$	per-step noise variance (the noise schedule)
$\boldsymbol{\alpha_t = 1 - \beta_t}$	per-step signal retention factor
$\bar{\alpha}_t = \prod_{i=1}^{t} \alpha_i$	cumulative signal retention factor
$\varepsilon \sim N(0, I)$	standard Gaussian noise, same shape as x_o

$$\beta_t \text{ increases with } t \Rightarrow \bar{\alpha}_t \text{ decreases with } t \text{ as } t \to T$$
$$\bar{\alpha}_T \approx 0 \Rightarrow x_T \approx N(0, I)$$

STEP 1

Forward process: a fixed Markov chain that adds noise

Each step keeps a shrinking fraction of the signal and injects a little Gaussian noise:

$$q(x_t \mid x_{t-1}) = \mathcal{N}\left(x_t;\ \sqrt{1-\beta_t}\, x_{t-1},\ \beta_t I\right)$$

Reparametrized:

$$x_t = \sqrt{1-\beta_t}\, x_{t-1} + \sqrt{\beta_t}\, \varepsilon_{t-1}, \qquad \varepsilon_{t-1} \sim \mathcal{N}(0, I)$$

- The forward process has no learnable parameters — it is fully specified by the schedule $\{\beta_t\}$.
- Because every step is Gaussian, the composition of steps stays Gaussian — which gives us a closed form (next slide).

STEP 1 · DERIVATION

The closed-form shortcut: jump to any time step

Unrolling two steps and using the fact that the sum of independent Gaussians is Gaussian:

$$x_t = \sqrt{\alpha_t}\, x_{t-1} + \sqrt{1-\alpha_t}\, \varepsilon_t \quad \Longrightarrow \quad x_{t-1} = \sqrt{\alpha_{t-1}}\, x_{t-2} + \sqrt{1-\alpha_{t-1}}\, \varepsilon_{t-2}$$

Substituting both sides and collecting the noise terms:

$$x_t = \sqrt{\alpha_t \alpha_{t-1}}\, x_{t-2} + \sqrt{\alpha_t(1-\alpha_{t-1}) + (1-\alpha_t)}\, \tilde{\varepsilon} \qquad \text{with} \quad \alpha_t(1-\alpha_{t-1}) + (1-\alpha_t) = 1 - \alpha_t\alpha_{t-1}$$

Repeating to x_0 gives the closed form used throughout DDPM:

$$x_t = \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1-\bar{\alpha}_t}\, \varepsilon, \qquad \bar{\alpha}_t = \prod_{i=1}^{t} \alpha_i, \qquad \varepsilon \sim \mathcal{N}(0, I)$$

STEP 1 · CONSEQUENCE

Why the closed form matters

- One sample for any t — no need to simulate t sequential steps during training.
- In practice we draw one random t per example per gradient step.
- x_t is a deterministic function of (x_0, ε), so the model can be trained with a single random draw.
- It makes the entire training objective computable in O(1) per example.

$$q(x_t \mid x_0) = \mathcal{N}\left(x_t;\ \sqrt{\bar{\alpha}_t}\, x_0,\ (1 - \bar{\alpha}_t) I\right)$$

$$t = T \quad \Longrightarrow \quad x_T \approx \varepsilon$$

Because x_t is available in closed form, the loss needs only one network evaluation per example.

STEP 2

Reverse process: the posterior we actually need

We want the inverse of the forward kernel — but Bayes' rule requires the unknown marginals $q(x_t)$:

$$q(x_{t-1} \mid x_t) = q(x_t \mid x_{t-1}) \frac{q(x_{t-1})}{q(x_t)}$$

Standard fix: condition on x_0 as well. Then all three terms are known Gaussian densities, so the posterior is Gaussian:

$$q(x_{t-1} \mid x_t, x_0) = q(x_t \mid x_{t-1}, x_0) \frac{q(x_{t-1} \mid x_0)}{q(x_t \mid x_0)}$$

This is the training target: an explicit Gaussian we can regress to.

STEP 2 · RESULT

The posterior is Gaussian — with known mean and variance

$$q(x_{t-1} \mid x_t, x_0) = \mathcal{N}\left(x_{t-1};\ \tilde{\mu}_t,\ \tilde{\sigma}_t^2 I\right)$$

Expanding the product of the three Gaussian densities and completing the square in x_{t-1}:

$$\tilde{\mu}_t = \frac{\sqrt{\bar{\alpha}_t}\,(1-\bar{\alpha}_{t-1})}{1-\bar{\alpha}_t}\,x_t \;+\; \frac{\sqrt{\bar{\alpha}_{t-1}}\,\beta_t}{1-\bar{\alpha}_t}\,x_0$$

$$\tilde{\sigma}_t^2 = \frac{\beta_t\,(1-\bar{\alpha}_{t-1})}{1-\bar{\alpha}_t}$$

The variance depends only on t — it is a known constant, not a quantity we must learn.

STEP 3

Variational bound: turning likelihood into a sum of KL terms

We maximise the data likelihood. Since it is intractable, we maximise its evidence lower bound:

$$-\log p_\theta(x_0) \;\leq\; -\log p_\theta(x_0) + D_{\mathrm{KL}}(q(x_{1:T} \mid x_0) \,\|\, p_\theta(x_{1:T} \mid x_0))$$

Writing the joint distributions along the Markov chain and rearranging gives a sum of three groups of terms:

$$-\log p_\theta(x_0) \;\leq\; \mathbb{E}_q[D_{\mathrm{KL}}(q(x_T \mid x_0) \,\|\, p_\theta(x_T))]$$

$$+ \sum_{t=2}^{T} D_{\mathrm{KL}}(q(x_{t-1} \mid x_t, x_0) \,\|\, p_\theta(x_{t-1} \mid x_t)) \;-\; \log p_\theta(x_0 \mid x_1)$$

KL Divergence measures the “difference between two distributions”. Here we use KL terms to measure the difference between the noise predicted by model and the actual noise added to the image.

STEP 3 · RESULT

From KL divergence to a simple noise-regression loss

Both distributions in L_{t-1} are Gaussian, so the KL has a closed form. With the variance fixed, only the means matter:

$$D_{\mathrm{KL}}(\mathcal{N}(\mu_1, \Sigma) \,\|\, \mathcal{N}(\mu_2, \Sigma)) = \frac{1}{2}(\mu_1 - \mu_2)^\top \Sigma^{-1}(\mu_1 - \mu_2)$$

Substituting $x_0 = (x_t - \sqrt{(1-\bar{\alpha}_t)}\cdot\varepsilon)/\sqrt{(\bar{\alpha}_t)}$ removes x_0 and leaves a prediction of the noise itself:

$$\mu_\theta(x_t, t) = \frac{1}{\sqrt{\alpha_t}}\left(x_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\varepsilon_\theta(x_t, t)\right)$$

$$\mathcal{L} = \mathbb{E}_{t,\, x_0,\, \varepsilon}\left[\left\| \varepsilon - \varepsilon_\theta\left(\sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1-\bar{\alpha}_t}\,\varepsilon,\ \ t\right)\right\|^2\right]$$

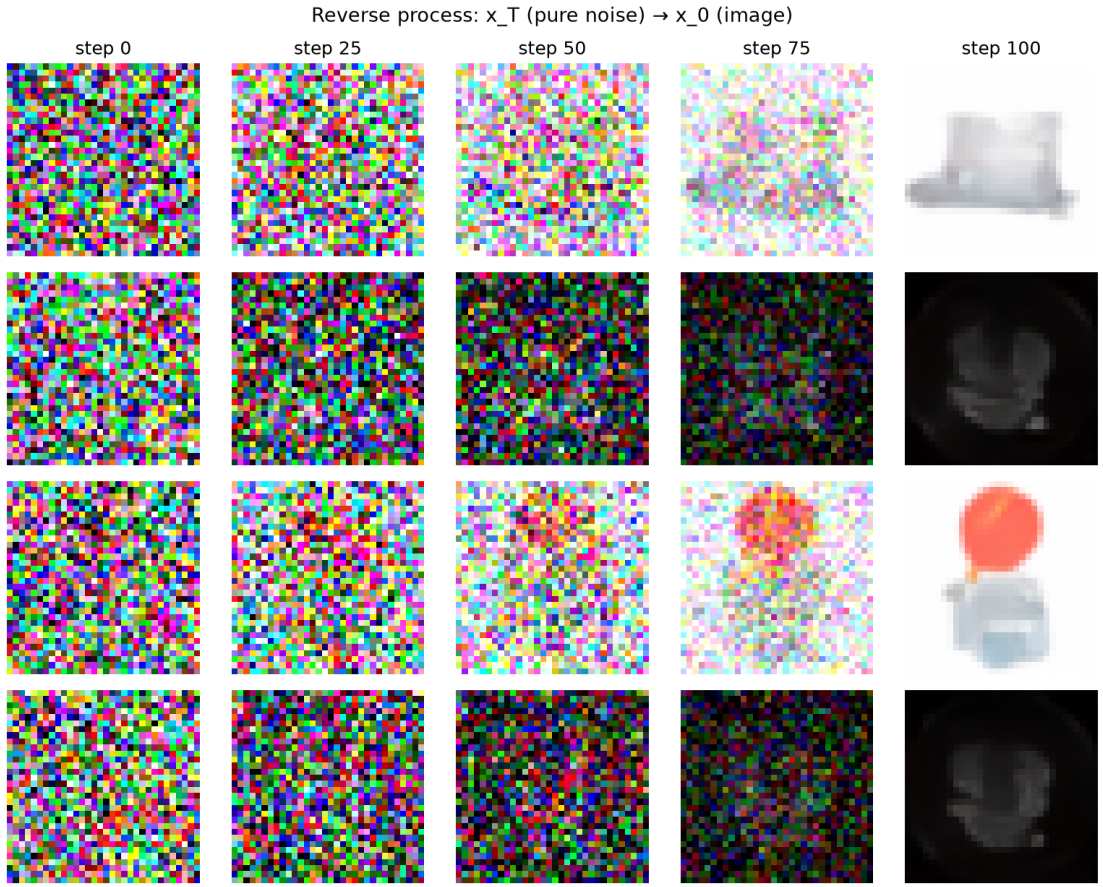
L2 loss shows the square of error between the added noise and the noise predicted by neural network.
Mean Squared Error(MSE) is the simplified training objective, depicting the accuracy for the model to predict noise.

STEP 4 · ALGORITHM 1

Training algorithm

```
repeat
    x0    ~ q(x0)                    # sample a clean image
    t     ~ Uniform({1, ..., T})      # pick a random time step
    eps   ~ N(0, I)                   # draw standard Gaussian noise
    gradient-descent step on:
        || eps - eps_theta( sqrt(abar[t]) * x0
                          + sqrt(1 - abar[t]) * eps, t ) ||^2
until converged
```

```
# cs231n/gaussian_diffusion.py  ->  p_losses()
t      = torch.randint(0, T, (b,))      # one random t per example
noise  = torch.randn_like(x_start)
x_t    = q_sample(x_start, t, noise)    # sqrt(abar)*x0 + sqrt(1-abar)*eps
target = noise if objective == 'pred_noise' else x_start
loss   = ((model(x_t, t) - target) ** 2).mean()
```



Reverse process $p(x_{t-1}|x_t)$: four independent runs from pure noise x_T down to x_0, using the pretrained weights.

STEP 4 · ALGORITHM 2

Sampling algorithm

```
x[T] ~ N(0, I)
for t = T, ..., 1 do
    z      = randn_like(x[t]) if t > 1 else 0
    x[t-1] = (1/sqrt(alpha[t])) * ( x[t]
               - (beta[t]/sqrt(1 - abar[t])) * eps_theta(x[t], t) )
             + sigma[t] * z
end for
return x[0]
```

```
# cs231n/gaussian_diffusion.py  ->  p_sample()
model_out = model(x_t, t)
x_start   = predict_start_from_noise(x_t, t, model_out)  # invert x_t
x_start.clamp_(-1.0, 1.0)        # keep generation stable
mean, std = q_posterior(x_start, x_t, t)
return mean + (t != 0) * std * torch.randn_like(x_t)
```

- start from pure noise;
- one net call per step;
- T = 1000, so sampling is slow;
- sigma[t]*z keeps it stochastic.

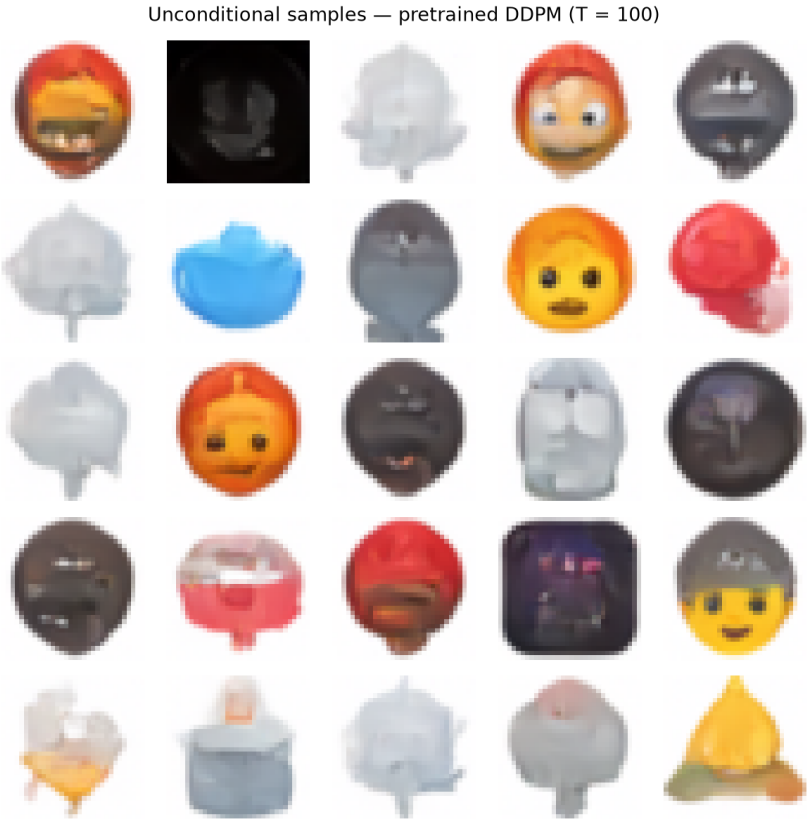
Design choices that matter

Noise schedule β_t	linear (original DDPM) · cosine (better SNR) · sigmoid (above 64×64)
Reverse variance σ_t^2	analytic value · DDPM fixes $\sigma_t^2 = \beta_t$ · iDDPM learns a hybrid
Network $\varepsilon\theta$	U-Net with skips · residual blocks · self-attention at low resolution
Timestep conditioning	sinusoidal embedding of t, injected by scale-and-shift

OUTCOME

Results and extensions

Sampled from the pretrained model



25 unconditional samples, T = 100.

- DDPM reported state-of-the-art FID on CIFAR-10 and high-quality 256×256 ImageNet samples.
- FID 3.17 on CIFAR-10 (unconditional) — competitive with the best GANs of its time.
- Quality improves with model size and with the number of sampling steps.

Direct descendants

– DDIM — deterministic, far fewer sampling steps

– Score-based SDE — continuous-time unification

– Classifier-free guidance — conditional control

– Latent diffusion — diffuse in a compressed latent space

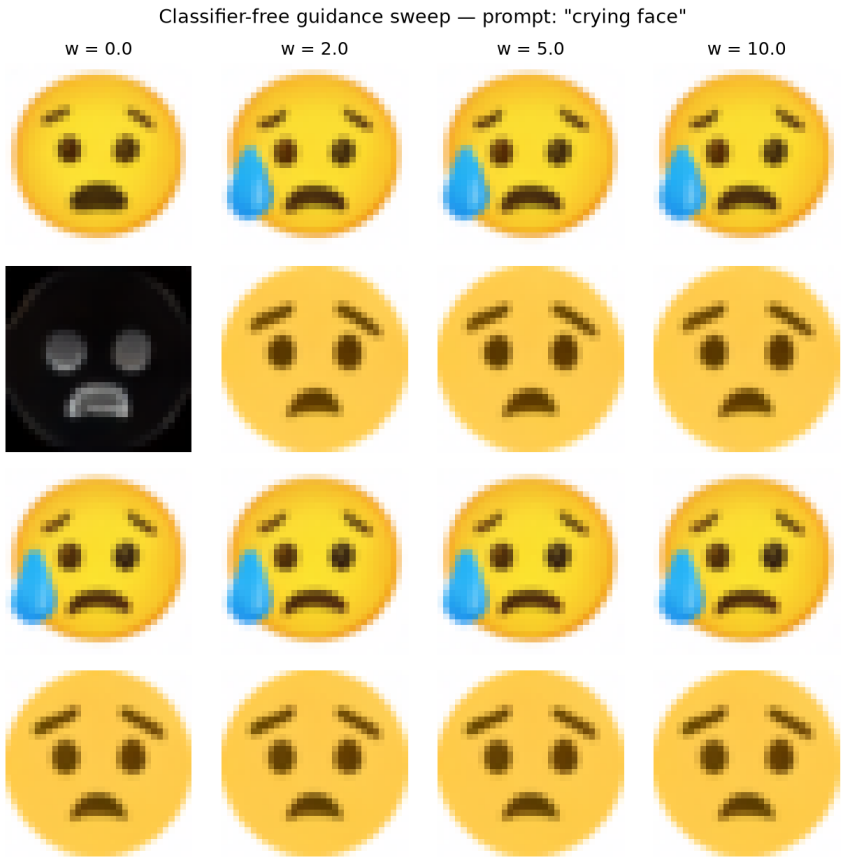
– Diffusion Transformers — replace U-Net with a Transformer

Generation results

Unconditional samples



Classifier-free guidance — "crying face" generation



`q_sample 5.1e-07` · `predict_start_from_noise 2.8e-06` · `predict_noise_from_start 1.1e-06`

U-Net 12.4M params · reproduced from the released pretrained checkpoint (step 70,000)

Takeaways

1. Diffusion replaces one hard generation problem with many easy denoising problems.
2. The forward process is fixed; a Gaussian closed form gives x_t directly from x_0.
3. The true reverse posterior is intractable, so we condition on x_0 to obtain a Gaussian.
4. The variational bound reduces to a simple noise-regression loss $L = E[\|\varepsilon - \varepsilon\theta\|^2]$.
5. Training is stable and parallel; the price is slow, iterative sampling.

SOURCES

References

[1] Ho, Jain & Abbeel. Denoising Diffusion Probabilistic Models. NeurIPS 2020. arXiv:2006.11239

[2] Sohl-Dickstein et al. Deep Unsupervised Learning using Nonequilibrium Thermodynamics. ICML 2015. arXiv:1503.03585

[3] Song & Ermon. Generative Modeling by Estimating Gradients of the Data Distribution. NeurIPS 2019. arXiv:1907.05600

[4] Song, Meng & Ermon. Denoising Diffusion Implicit Models (DDIM). ICLR 2021. arXiv:2010.02502

[5] Song et al. Score-Based Generative Modeling through SDEs. ICLR 2021. arXiv:2011.13456

[6] Nichol & Dhariwal. Improved Denoising Diffusion Probabilistic Models. ICML 2021. arXiv:2102.09672

[7] Ho & Salimans. Classifier-Free Diffusion Guidance. 2022. arXiv:2207.12598

[8] Rombach et al. High-Resolution Image Synthesis with Latent Diffusion Models. CVPR 2022. arXiv:2112.10752

[9] Peebles & Xie. Scalable Diffusion Models with Transformers (DiT). ICCV 2023. arXiv:2212.09748

[10] Liu, Gong & Liu. Flow Straight and Fast: Learning to Generate and Transfer Data with Rectified Flow. 2022. arXiv:2209.03003

All references verified against the official proceedings / arXiv listings.

Diffusion Models



Thank you for watching !

Questions are welcome.

Presented by:
WU Peihang
24108869D